

# LangGraph

## ■ LangGraph

- LLM 기반의 Agent와 복잡한 AI 워크플로를 그래프(Graph) 구조로 설계하고 실행하는 Python 프레임워크

| LangChain                | LangGraph               |
|--------------------------|-------------------------|
| 단순한 Chain 구성             | 복잡한 Agent Workflow      |
| 직선형 실행                   | 그래프 기반 실행               |
| 상태 관리 제한                 | 상태(State) 관리 내장         |
| 반복 처리 어려움                | Loop 지원                 |
| 조건 분기 제한                 | Conditional Edge 지원     |
| Human-in-the-loop 구현 어려움 | Human-in-the-loop 기본 지원 |
| 장기 실행에는 부적합              | 장기 실행(Long-running) 지원  |

LangChain이 LLM을 연결(Chain) 하는 데 중점을 둔다면, LangGraph는 상태(State)를 유지하면서 여러 단계의 작업, 반복(Loop), 조건 분기, 멀티 에이전트, 사람의 승인(Human-in-the-Loop) 등을 안정적으로 수행할 수 있도록 설계되었습니다

## ■ LangGraph

- 순환 그래프 모델을 지원해 재시도, 자기수정, 반복적 추론에 필수적인 '이전 노드로 되돌아가는' 흐름을 만들 수 있습니다
- 에이전트 장애의 60% 이상이 상태 관리 실패(맥락 손실, 단계 반복, 복구 불가능한 크래시 등)에서 발생되며 이 문제를 해결하기 위해 LangGraph가 설계되었습니다

| 요소         | LangGraph                                                                                                                              |
|------------|----------------------------------------------------------------------------------------------------------------------------------------|
| State (상태) | 모든 노드가 읽고 쓸 수 있는 공유 TypedDict(또는 Pydantic 모델) 객체<br>에이전트의 "기억" 역할을 합니다.<br>함수 인자로 컨텍스트를 계속 넘길 필요 없이, 하나의 공유된 "노트"에 각 단계가 정보를 기록하고 읽습니다 |
| Node       | LLM 호출, Tool 호출, API 호출, Database 조회, Retriever 검색 등 하나의 작업을 수행하는 함수                                                                   |
| Edge       | 노드 간 흐름을 정의<br>조건에 따라 다음 노드를 결정하는 조건부 엣지(conditional edge)도 지원                                                                         |
| Graph      | 모든 Node와 Edge를 모은 것                                                                                                                    |

# LangGraph

## ■ LangGraph

- Tool을 여러 번 반복 호출
- 조건에 따라 다른 Tool 실행
- 실패하면 재시도
- 사용자 승인 후 계속 진행
- 여러 Agent가 협업
- 중간 상태 저장 및 재시작

LangGraph는 저수준 실행 엔진이고, LangChain은 그 위에 올라가는 컴포넌트 라이브러리입니다. LangChain은 빠르게 에이전트를 만들 수 있는 고수준 추상화를 제공하고, LangGraph는 세밀한 제어가 필요한 복잡한 워크플로우를 위한 것이며, LangChain 에이전트도 LangGraph 위에서 동작합니다.

## ■ LangGraph

| 요소               | LangGraph                                                             |
|------------------|-----------------------------------------------------------------------|
| State            | 그래프 전체에서 공유되는 데이터<br>Agent의 기억(Memory)                                |
| Node             | 하나의 작업을 수행하는 함수<br>LLM 호출, Tool 호출, API 호출, Database 조회, Retriever 검색 |
| Edge             | Node와 Node를 연결                                                        |
| Conditional Edge | 조건에 따라 실행 경로를 변경                                                      |
| Graph            | 모든 Node와 Edge를 모은 것                                                   |
| Compile          | 그래프를 실행 가능한 객체로 변환                                                    |

사용자 질문

Graph.invoke()

State 생성

Node 실행

State 업데이트

다음 Node

최종 결과

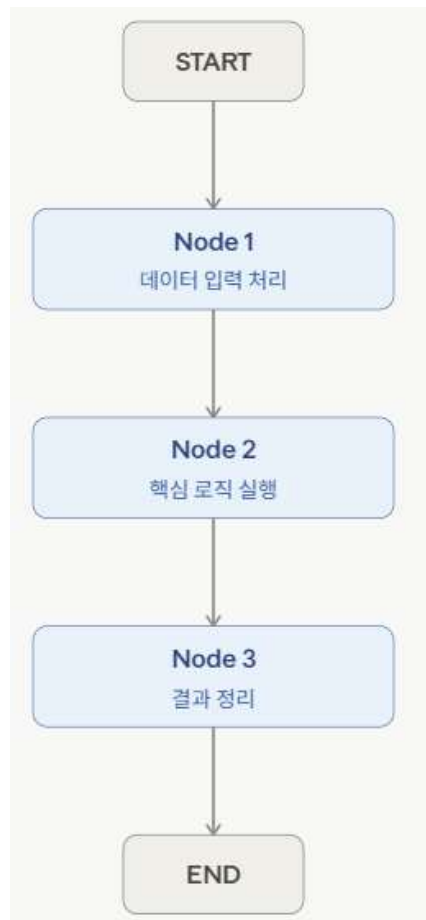
## ■ LangGraph 적합 APP

- 여러 단계를 거치는 복잡한 에이전트 워크플로우
- 재시도/반복이 필요한 로직
- 장시간 실행되거나 중단 후 재개가 필요한 작업
- 여러 에이전트가 협업하는 멀티 에이전트 시스템
- 감사(audit)나 디버깅을 위해 흐름을 명시적으로 시각화해야 하는 경우

# LangGraph

## ■ LangGraph

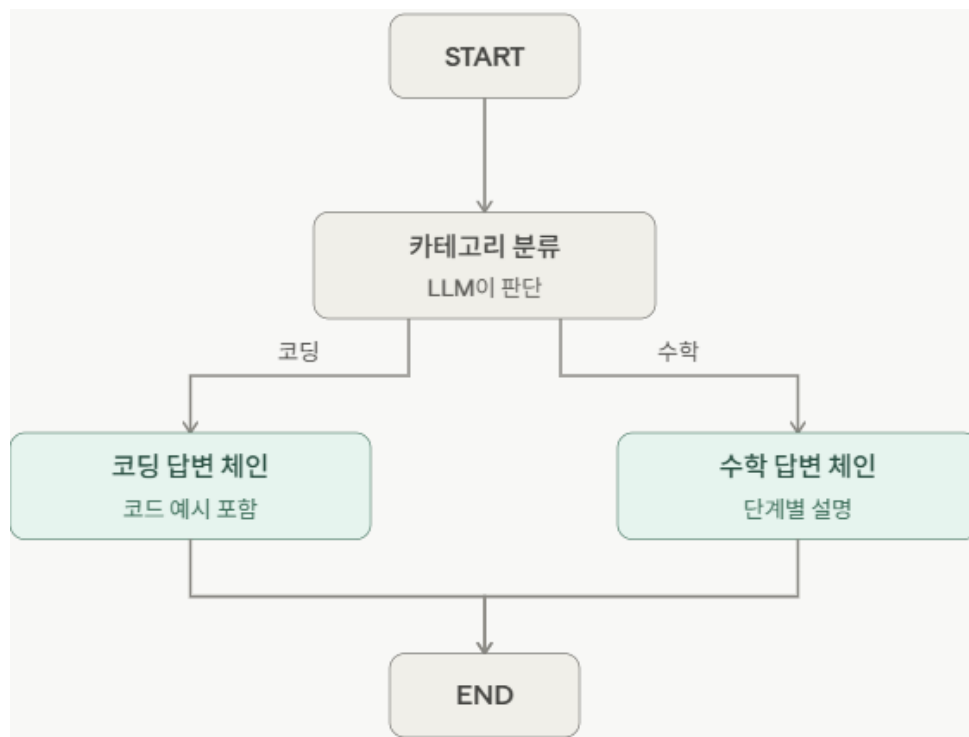
- StateGraph에 노드를 순서대로 연결하고 add\_edge()로 이어 붙이는 가장 단순한 형태입니다.
- 각 노드는 state를 받아 일부를 업데이트해서 다음 노드로 넘깁니다.



# LangGraph

## ■ LangGraph

- `add_conditional_edges()`를 사용해서, 라우터 함수의 반환값에 따라 다음에 실행할 노드를 다르게 선택하는 구조입니다. 카테고리 분류 후 그에 맞는 체인으로 갈라지는 패턴





# LangGraph

## ■ LangGraph

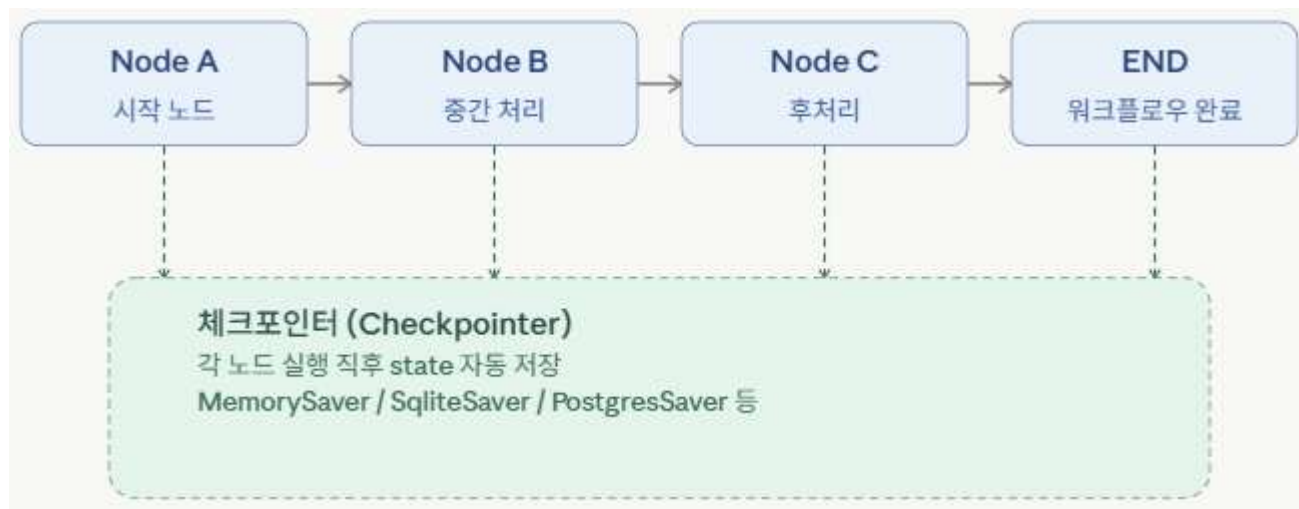
- 하나의 노드에서 "조건 충족 여부"를 판단해서, 충족되지 않으면 자기 자신으로 돌아가고(self-loop), 충족되면 다음 단계로 넘어가는 구조
- loop workflow : `add_conditional_edges()` (자기 자신을 타겟으로)



# LangGraph

## ■ LangGraph

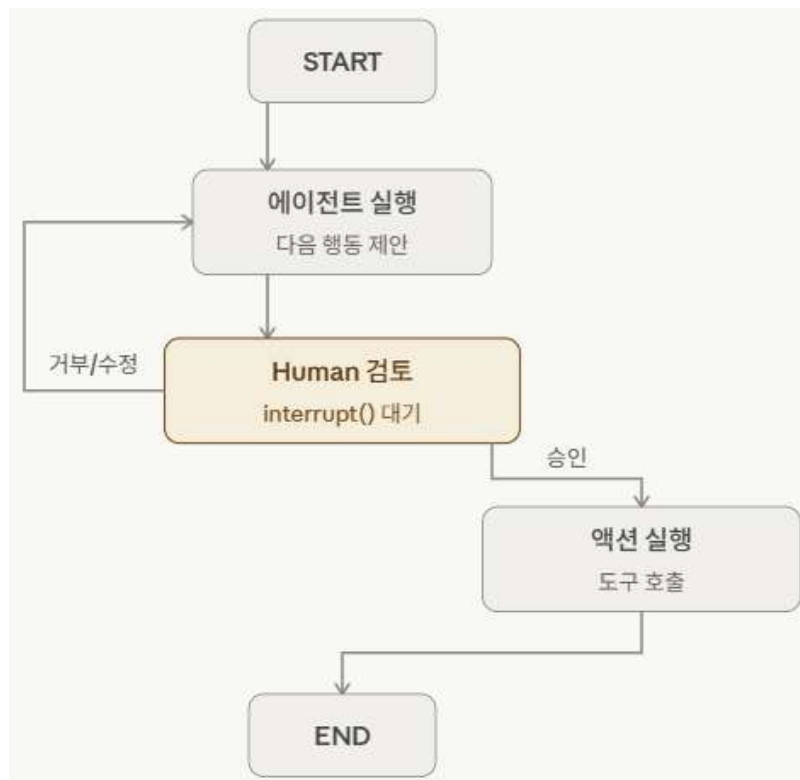
- checkpointer(예: MemorySaver, SqliteSaver, PostgresSaver)를 compile()에 연결하면, LangGraph가 각 노드 실행 직후마다 자동으로 state 스냅샷을 저장합니다.
- 중간에 프로세스가 끊겨도 같은 thread\_id로 이어서 재개할 수 있습니다.
- compile(checkpointer=...)



# LangGraph

## ■ LangGraph

- `interrupt()`로 그래프 실행을 일시 정지시키고 사람의 승인/수정을 기다린 뒤, 승인되면 다음 단계로 진행하고 거부되면 에이전트가 다시 시도하는 구조
- 결제, 이메일 발송처럼 되돌릴 수 없는 액션 전에 자주 사용됩니다.



# LangGraph

## ■ LangGraph

```
from typing import TypedDict
from langgraph.graph import StateGraph, START, END
```

```
class State(TypedDict):
    message: str
```

```
def hello(state: State):
    return {
        "message": "Hello LangGraph"
    }
```

```
builder = StateGraph(State)
```

```
builder.add_node("hello", hello)
builder.add_edge(START, "hello")
builder.add_edge("hello", END)
```

```
graph = builder.compile()
result = graph.invoke( {"message":""} )
print(result)
```